CO1-AT3-Coding Chellenge

View Only

← Back

QUESTIONS

Q1
Smart University Payroll Management System
25 marks

Q2
Smart Vehicle Insurance Claim System
25 marks

2/2 answered

Q1 **Smart University Payroll Management System** 25 marks

A university employs three categories of staff: Teaching Faculty, Technical Staff, and Visiting Faculty. The payroll system should calculate the monthly salary based on the employee type while maintaining a common employee hierarchy. Requirements Create a superclass Employee with the following private data members: employeeId employeeName department basicSalary Provide: Default constructor Parameterized constructor Getter and Setter methods displayEmployee() calculateSalary() Create the following subclasses: TeachingFaculty teachingHours incentivePerHour Salary: basicSalary + (teachingHours × incentivePerHour) TechnicalStaff overtimeHours overtimeRate Salary: basicSalary + (overtimeHours × overtimeRate) VisitingFaculty lecturesHandled paymentPerLecture Salary: lecturesHandled × paymentPerLecture Override calculateSalary() in all subclasses. Create an array of Employee objects and demonstrate runtime polymorphism. Display: Employee details Monthly salary Highest-paid employee Average salary Search an employee using the employee ID.

Test Case 1 (Normal Case)

Input

3

TeachingFaculty

101

Dr. Arun

CSE

50000

20

500

TechnicalStaff

102

Mr. Ravi

ECE

35000

15

300

VisitingFaculty

103

Dr. Kumar

IT

0

40

1200

Search Employee ID:

102

Expected Output

Employee Details

Employee ID : 101

Name : Dr. Arun

Department : CSE

Monthly Salary : 60000.0

Employee ID : 102

Name : Mr. Ravi

Department : ECE

Monthly Salary : 39500.0

Employee ID : 103

Name : Dr. Kumar

Department : IT

Monthly Salary : 48000.0

Highest Paid Employee

Employee ID : 101

Salary : 60000.0

Average Salary

49166.67

Employee Found

Employee ID : 102

Name : Mr. Ravi

Salary : 39500.0

Your Code

```
import java.util.Scanner;
public class Main{
    public static void main(String[] args)
        Scanner sc=new Scanner(System.in);
        int n=Integer.parseInt(sc.nextLine
        Employee[] emp=new Employee[n];
        for(int i=0;i<n;i++){
            String type=sc.nextLine().trin
            while(type.isEmpty())
                type=sc.nextLine().trim();
            int id=Integer.parseInt(sc.nex
            String name=sc.nextLine().trin
            String dept=sc.nextLine().trin
            double basicSalary=Double.pars
            int value1=Integer.parseInt(sc
            double value2=Double.parseDoul
            if(type.equalsIgnoreCase("Teac
                emp[i]=new TeachingFaculty
            }
            else if(type.equalsIgnoreCase
                emp[i]=new TechnicalStaff
            }
            else if(type.equalsIgnoreCase
                emp[i]=new VisitingFaculty
            }
        }
        String temp=sc.nextLine().trim();
        while(temp.isEmpty())
```

```
            temp=sc.nextLine().trim();
        int searchId;
        if(temp.equalsIgnoreCase("Search I
            searchId=Integer.parseInt(sc.n
```



```
            System.out.println("Department
            System.out.println("Monthly Sa
            System.out.println();
            total+=e.calculateSalary();
            if(e.calculateSalary()>highest
                highest=e;
        }
        System.out.println("Highest Paid E
        System.out.println("Employee ID :
        System.out.println("Salary : "+hig
        System.out.println();
        System.out.println("Average Salary
        System.out.printf("%.2f\n",(total,
        System.out.println();
        boolean found=false;
        for(Employee e: emp){
            if(e.getEmployeeId()==searchId
                found=true;
                System.out.println("Employ
                System.out.println("Employ
                System.out.println("Name  :
                System.out.println("Salary
                break;
            }
        }
        if(!found){
            System.out.println("Employee N
        }
        sc.close();
    }
}
class Employee{
    private int employeeId;
    private String employeeName;
```

```
    private String department;
    private double basicSalary;
    Employee(int employeeId,String employe
        this.employeeId=employeeId;
        this.employeeName=employeeName;
        this.department=department;
        this.basicSalary=basicSalary;
    }
    public int getEmployeeId(){
        return employeeId;
    }
    public String getEmployeeName(){
        return employeeName;
    }
    public String getDepartment(){
        return department;
    }
    public double getBasicSalary(){
        return basicSalary;
    }
    public double calculateSalary(){
        return basicSalary;
    }
}
class TeachingFaculty extends Employee{
    int teachingHours;
    double incentivePerHour;
    TeachingFaculty(int id,String name,St
        super(id,name,dept,salary);
        teachingHours=h;
        incentivePerHour=p;
    }
    public double calculateSalary(){
        return getBasicSalary()+(teaching
    }
}
class TechnicalStaff extends Employee{
    int overtimeHours;
    double overtimeRate;
    TechnicalStaff(int id,String name,Str
        super(id,name,dept,salary);
        overtimeHours=h;
        overtimeRate=r;
    }
    public double calculateSalary(){
        return getBasicSalary()+(overtime
    }
}
```

```
class VisitingFaculty extends Employee{
    int lecturesHandled;
    double paymentPerLecture;
    VisitingFaculty(int id,String name,Str
        super(id,name,dept,salary);
        lecturesHandled=l;
        paymentPerLecture=p;
    }
    public double calculateSalary(){
        return lecturesHandled*paymentPerL
    }
}
```

Input

```
3
TeachingFaculty
101
Dr.Arun
```

Output

```
Employee Details

Employee ID : 101
Name : Dr.Arun
Department : CSE
Monthly Salary : 60000.0
```

Submitted